

Тема 10. Проектиране на интерфейси едновременно за няколко устройства. Responsive design. Съвременни средства за реализация на потребителски интерфейси (React, Vue, Angular).

I. Проектиране на интерфейси за множество устройства	4
A. Определение на проектирането на интерфейси за множество устройства	4
B. Кои са четирите C-та в дизайна на потребителския интерфейс (UX)?.....	4
Съгласуваност (Consistency):	5
Постоянство (Continuity):	5
Контекст (Context):.....	5
Допълващ (Complementary):	5
C. Какво е Multi-Device Experience?	5
D. Основни принципи	5
1) Идентифициране на основното потребителско изживяване	5
2) Идентифициране на групите устройства за вашия продукт	6
3) Адаптиране на изживяването за всеки контекст на употреба	7
4) Дизайн за най-малкия екран първо	8
5) Не забравяйте за големите екрани	9
6) Предоставете консистентно изживяване	10
7) Създайте непрекъснато изживяване	11
8) Тествайте Вашия Дизайн	11
II. Responsive design	12
A. Какво представлява Responsive Design?	12
B. Основни принципи.....	12
1) Fluid Grid System.....	13
2) Fluid Image Use	14
3) Media Queries	15

C. Най-добри практики	16
? Мобилен подход (Mobile-First Approach):.....	16
? Използвайте CSS Frameworks	16
? Респонсивни единици.....	16
? Мащабируема типография	16
? Медийни заявки	16
? Viewport Meta Tag.....	17
? Гъвкави изображения	17
? Приоритизиране на съдържанието	17
? Тестване	17
? Оптимизация на производителността:.....	17
? Потребителско изживяване	17
IV. Съвременни средства за реализация на потребителски интерфейси	17
A. React - общ преглед, основни принципи и примери за използване	17
1) Общ преглед.....	17
2) Основни принципи	18
? Virtual DOM	18
? Компоненти:	19
? Състояние (State):.....	20
? Пропс (Props):	20
? Жизнен цикъл (Lifecycle):	21
3) Примери.....	22
B. Vue - общ преглед, основни принципи и примери за използване	23
1) Общ преглед.....	23
2) Основни принципи	23
Принцип на единичната отговорност - Single Responsibility Principle (SRP) :	23
Принцип на отворената/затворената функционалност - Open/Closed	
Principle (OCP):	23
Принцип на заместваемостта на Лисков - Liskov Substitution Principle (LSP):	24
Принцип на разделянето на интерфейса - Interface Segregation Principle (ISP)	
.....	24

Принцип на инверсията на зависимостта - Dependency Inversion Principle (DIP):	24
3) Примери.....	24
C. Angular - общ преглед, основни принципи и примери за използване.....	25
1) Общ преглед.....	25
2) Основни принципи	25
Принцип на единичната отговорност (Single Responsibility Principle - SRP) ..	25
Принцип на отвореност/затвореност (Open/Closed Principle - OCP)	26
Принцип на подстановяването на Лисков (Liskov Substitution Principle - LSP)	26
Принцип на разделянето на интерфейса (Interface Segregation Principle - ISP)	26
Принцип на инверсията на зависимостта (Dependency Inversion Principle - DIP)	26
3) Примери.....	26

I. Проектиране на интерфейси за множество устройства

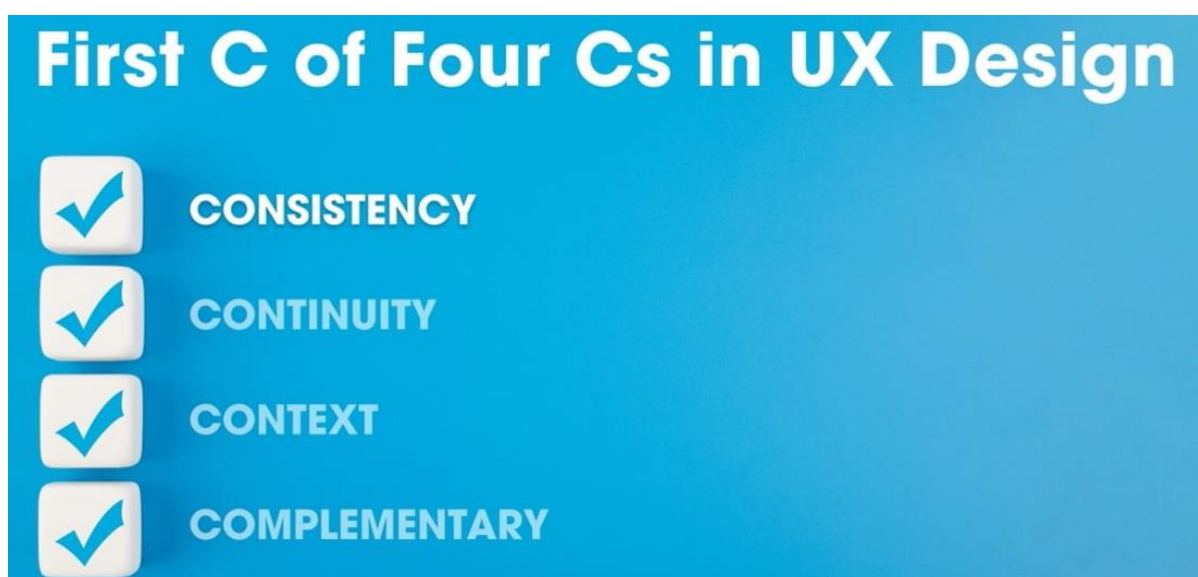
A. Определение на проектирането на интерфейси за множество устройства

Проектирането на интерфейси за множество устройства е процесът на създаване на потребителски интерфейси, които са оптимизирани за визуализация и функционалност на различни устройства, като например компютри, таблети и мобилни телефони. Този процес включва адаптиране на дизайна, структурата и функционалността на интерфейса, за да осигури консистентно и удобно изживяване на потребителите независимо от тяхното устройство и размер на екрана.



B. Кои са четирите C-та в дизайна на потребителския интерфейс (UX)?

Четирите C-та в дизайна на потребителския интерфейс (UX) са:



Съгласуваност (Consistency):

Съгласуваността се отнася до постоянството на дизайна и визията на интерфейса на потребителското приложение. Това включва използването на еднакви цветове, шрифтове, разположение на елементи и интерактивни ефекти в целия интерфейс, което прави потребителското изживяване по-прогнозируемо и удобно.

Постоянство (Continuity):

Постоянството се отнася до плавното преживяване на потребителите при преминаването между различните части и функционалности на приложението. Важно е потребителите да имат усещане за единство и последователност в потребителския интерфейс, което ги улеснява в навигацията и използването на приложението.

Контекст (Context):

Контекстът означава адаптацията на интерфейса към специфични ситуации или нужди на потребителите. Това включва вземането предвид на контекстуалните фактори като местоположение, време на деня, предпочитания на потребителите и други, за да се предостави персонализирано и релевантно изживяване.

Допълващ (Complementary):

Допълнителността се отнася до добавянето на елементи и функции, които подпомагат и допълват основните цели и задачи на потребителския интерфейс. Тези елементи могат да включват интуитивни икони, контекстуални подсказки, справочни материали и други, които улесняват потребителите в използването на приложението и подобряват техния UX.

C. Какво е Multi-Device Experience?

Мулти-устройствено изживяване означава създаване на потребителски интерфейси, които се адаптират към различни устройства, като например настолни компютри, лаптопи, таблети и мобилни телефони. Важността на проектирането на мулти-устройствени изживявания е очевидна, тъй като потребителите вече не се доверяват само на стационарен компютър или мобилно устройство. Те ще използват и двата все по-често, прехвърляйки се безпроблемно между различни контексти и среди.

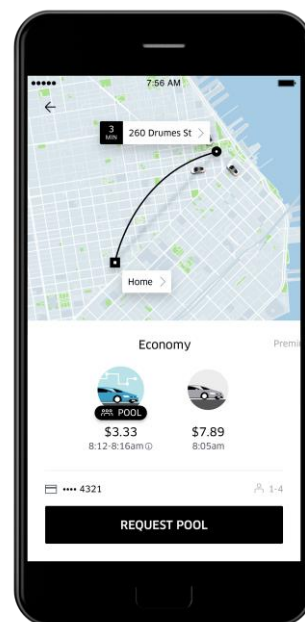
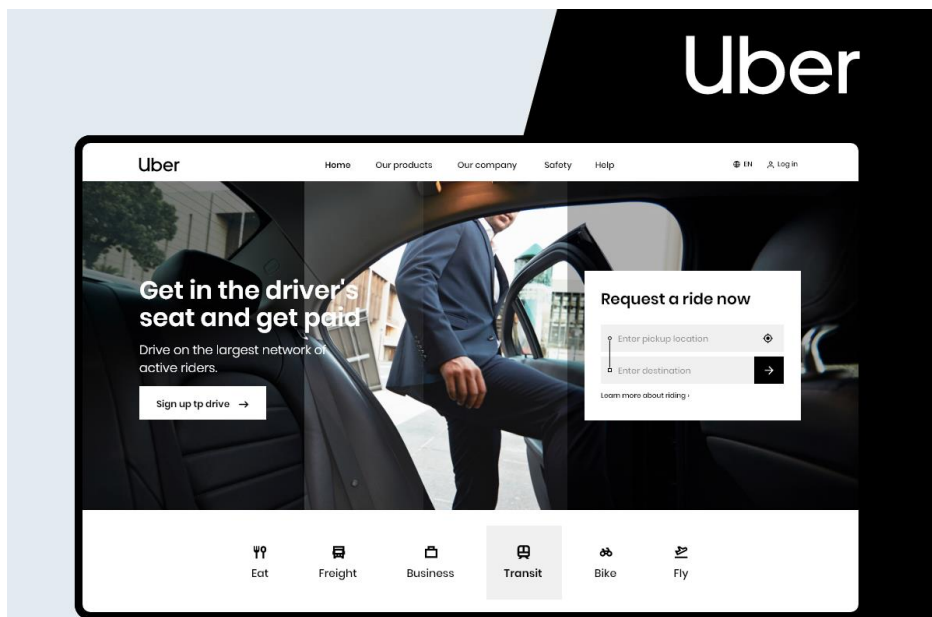
D. Основни принципи

1) Идентифициране на основното потребителско изживяване

Всеки продукт има основно потребителско изживяване, което основно е причината за съществуването му. То решава проблем, който хората имат, и чрез това става смислено и предоставя стойност за тях. Комбинацията от ключово съдържание и функции представлява основното потребителско изживяване. За да откриете основното потребителско изживяване на вашия продукт, задайте си въпроса: *"Кои са най-честите и важни задачи, които клиентите трябва да изпълнят?"* Той е от съществено значение, за да се поддържат тези основни задачи на всеки канал, който използвате за вашия продукт.

Проектиране на потребителски интерфейси

Например, основното потребителско изживяване на Uber е лесното заявяване на шофьор, по всяко време. Тази функционалност трябва да функционира добре на всяко устройство, предназначено за тази цел, независимо от големината на екрана.



2) Идентифициране на групите устройства за вашия продукт

Въпреки че на пазара има безброй устройства с различни размери на екрана и е абсолютно невъзможно да се насочат индивидуални устройства, е възможно да се определят групи устройства за вашия продукт, базирани на задачите, върху които потребителят вероятно ще се фокусира. Най-честите групи устройства са:

- Мобилни телефони
- Таблети
- Десктоп компютри
- Смарт телевизори
- Смарт часовници



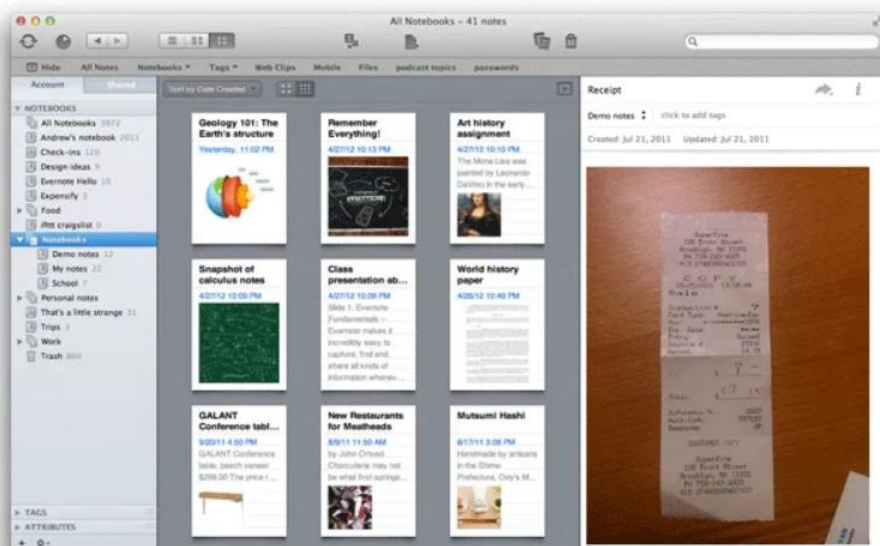
Различните групи устройства предоставят различни услуги в различни контексти: потребителите се ангажират с различни режими на взаимодействие в зависимост от типа на екрана, който гледат. Например, мобилните телефони се използват главно за микро-задачи и имат кратки потребителски сесии. Таблетите се използват предимно за консумация на съдържание и понастоящем не се считат за работен инструмент за повечето хора. Разбирането на този основен контекстуален аспект за различните типове устройства е съществено за изграждането на добро потребителско изживяване.

3) Адаптиране на изживяването за всеки контекст на употреба

След като идентифицирате основното потребителско изживяване на вашия продукт и изберете набор от групи устройства, които бихте искали да подкрепите, трябва да адаптирате изживяването за всяка група (за всеки контекст на употреба). Проектирането за контекста е особено важно, когато проектирате за различни групи устройства.

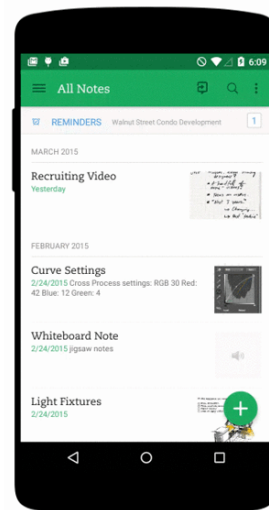
Първо, не всички функции имат смисъл на всички устройства. Трябва да идентифицирате различните сценарии, в които продуктът ви ще бъде използван сред различните групи устройства, и да проектирате изживяване, подходящо за всеки от тези сценарии. Например, често мобилните потребители искат различни неща от продукта в сравнение с потребителите на десктоп устройства. Вземете Evernote, популярен продукт за записване на бележки, който е на разположение на множество устройства, за пример. Неговата десктоп версия е оптимизирана за консумиране на съдържание:

Evernote приложението за десктоп е оптимизирано за четене на текст и преглед на медийни файлове.



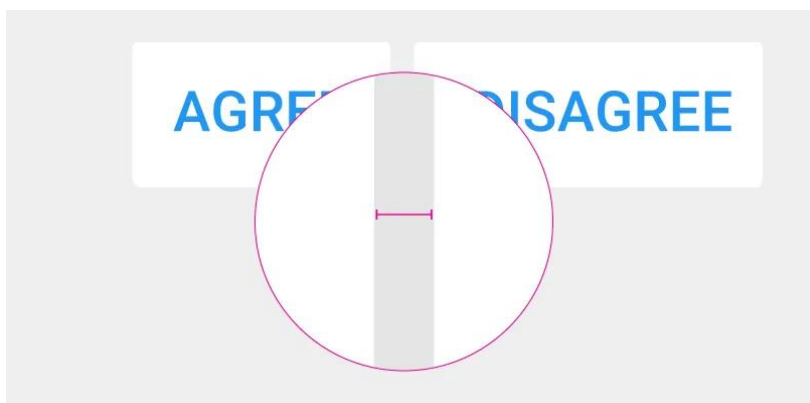
Докато мобилната версия е оптимизирана за записване на бележки, снимки и записване на аудио:

Evernote разбира мобилния контекст: използва възможностите на устройството и осигурява бърз начин за записване на идеи (добавяне на текстова бележка, заснемане на снимка или задаване на напомняне).



Второ, различните екрани позволяват използването на различни методи за въвеждане. Разгледайте докосването като пример. Някои общи грешки, които дизайнерите правят при проектиране за устройства с докосване (мобилни телефони и смарт часовници), включват:

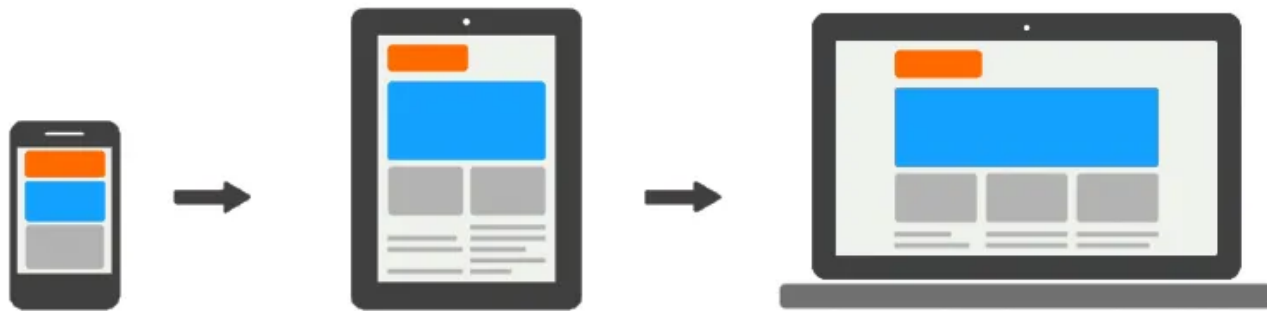
- **Малки цели за докосване.** Целите за докосване (като бутони за действие) трябва да бъдат адекватно големи. Минимум 7 мм обикновено е достатъчно, въпреки че по-добре е да се използва размер на целта за докосване от 10 мм.
- **Поставяне на елементи твърде близо един до друг.** Трябва да се има предвид размерът на целите за докосване, както и разстоянието между тях, като разстоянието помага да се отдели контролите и да се осигури пространство на потребителския интерфейс. Препоръчителното разстояние за предотвратяване на грешки при въвеждане е минимум 23 точки.



- **Използване на състоянието "hover"** - При докосване на екрани, няма "hover".

4) Дизайн за най-малкия екран първо

Исторически дизайнерите винаги са работили от най-големия екранен дизайн към най-малкия - което означава, че първоначалният и основен дизайн беше за пълната десктоп версия (тя имаше най-голямата функционалност). Само след като десктоп дизайнът беше завършен, той беше пренасян на мобилния и другите групи устройства. Въпреки това, когато проектираме за десктоп, обикновено се сблъскваме с проблем: към продукта се добавят много функции, особено когато са включени множество заинтересовани страни. Това не е изненадващо, тъй като добавянето на функции е относително лесно, когато разполагате с много пространство. Практическият опит ясно показва, че е по-добре да проектирате с подхода Mobile First и да създадете приложението си с най-малкия екран, който е релевантен за вашите потребители.



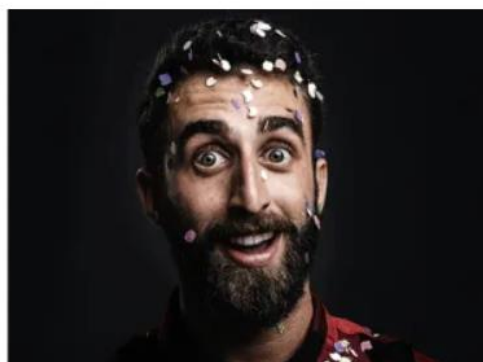
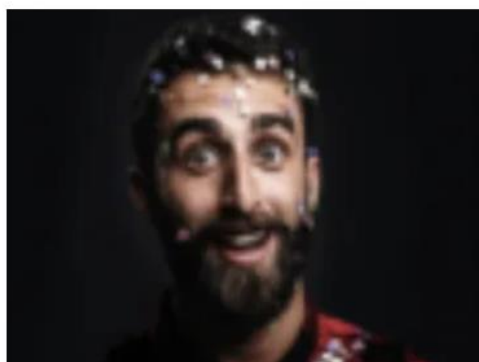
Когато проектират първо за най-малкия релевантен размер на екрана, ви принуждава да решите какво е най-важно. След известно време ще приложите същия подход на внимателен подбор и за другите версии на продукта, било то на десктоп, таблет или телевизор.

В повечето случаи телефонът ще бъде най-малкият релевантен размер на екрана. Ако устройствата за носене са важни за вас, тогава трябва да вземете предвид и микроекраните с още по-малки резолюции.

5) Не забравяйте за големите екрани

Мислете и за големите екрани, както и за малките, и отделете на големите екрани същото внимание, което отделяте на малките:

- Не просто мащабирайте дизайна си, за да се побере на големите екрани. Възползвайте се напълно от допълнителното пространство, което имате на разположение.



Left: Low quality image. Right: Correct resolution.

- Уверете се, че изображенията не загубват качество, докато се увеличават за най-големите екрани.

- Обмислете специфичните особености на големи екрани. Всяка група устройства има свои специфични характеристики. Например, проектирането за телевизионни екрани е известно като "проектиране за 10-футово изживяване", защото от разстоянието на дивана, видимият размер на елементите на екрана е забележимо по-малък в сравнение с настолния компютърен екран.



6) Предоставете консистентно изживяване

Консистентното изживяване означава, че приложението и неговото изживяване са подобни във всички размери на екрана. Консистентното потребителско изживяване, независимо от устройството, е един от ключовите компоненти на успешното потребителско изживяване на многоканалните потребители:

- То задава очаквания за бъдещите взаимодействия с вашия продукт и изгражда доверие на потребителя.
- Консистентните изживявания правят по-лесно за потребителите, които са взаимодействали с вашия продукт на други устройства.

Вместо да приспособявате дизайните си към всяко едно от все повече нарастващите числа на екрани и устройства, можете да ги разглеждате като различни страни на едно и също изживяване. Например приложението Google Search предоставя същото търсене на всички устройства. Когато дизайните и функциите са консистентни, потребителите могат да изпълняват задачи по-бързо и по-ефективно на устройството по тяхно избор.

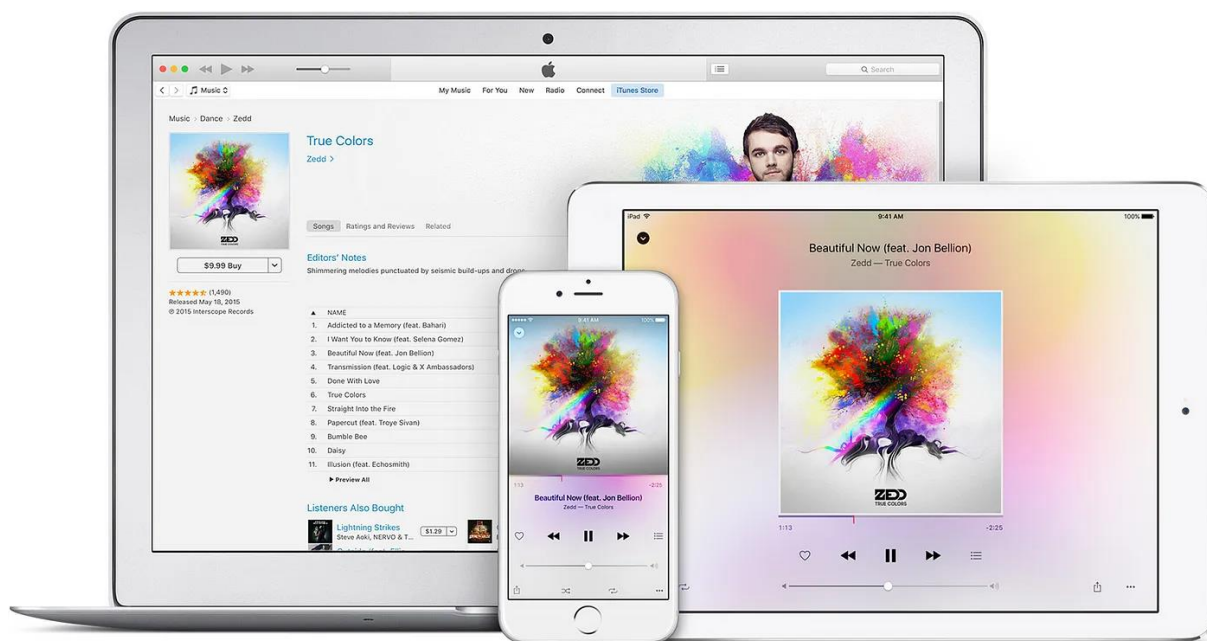


Проектиране на потребителски интерфейси

7) Създайте непрекъснато изживяване

Създаването на непрекъснато изживяване в различните групи устройства е много важно за вашите потребители. Хората свободно се движат от едно устройство на друго, за да свършат нещата си, и когато преминават от едно устройство на друго, те очакват техните продукти и услуги да преминават с тях. Това означава, че потребителите не трябва да мислят за устройството, което използват, промените в средата или промените в контекста и могат да разчитат на отлична функционалност и лесна употреба независимо от устройството.

На база на сценариите за използване, може да желаете да осигурите, че консумацията на съдържание на всяко устройство е синхронизирана. Вземете за пример Apple Music: можете да настроите плейлист на вашия Mac и той незабавно ще бъде достъпен на вашия iPhone, или можете да започнете да слушате песента на вашия iPhone, и когато преминете към десктопа, ще бъдете върнати в точката, където сте били на iPhone.



8) Тествайте Вашия Дизайн

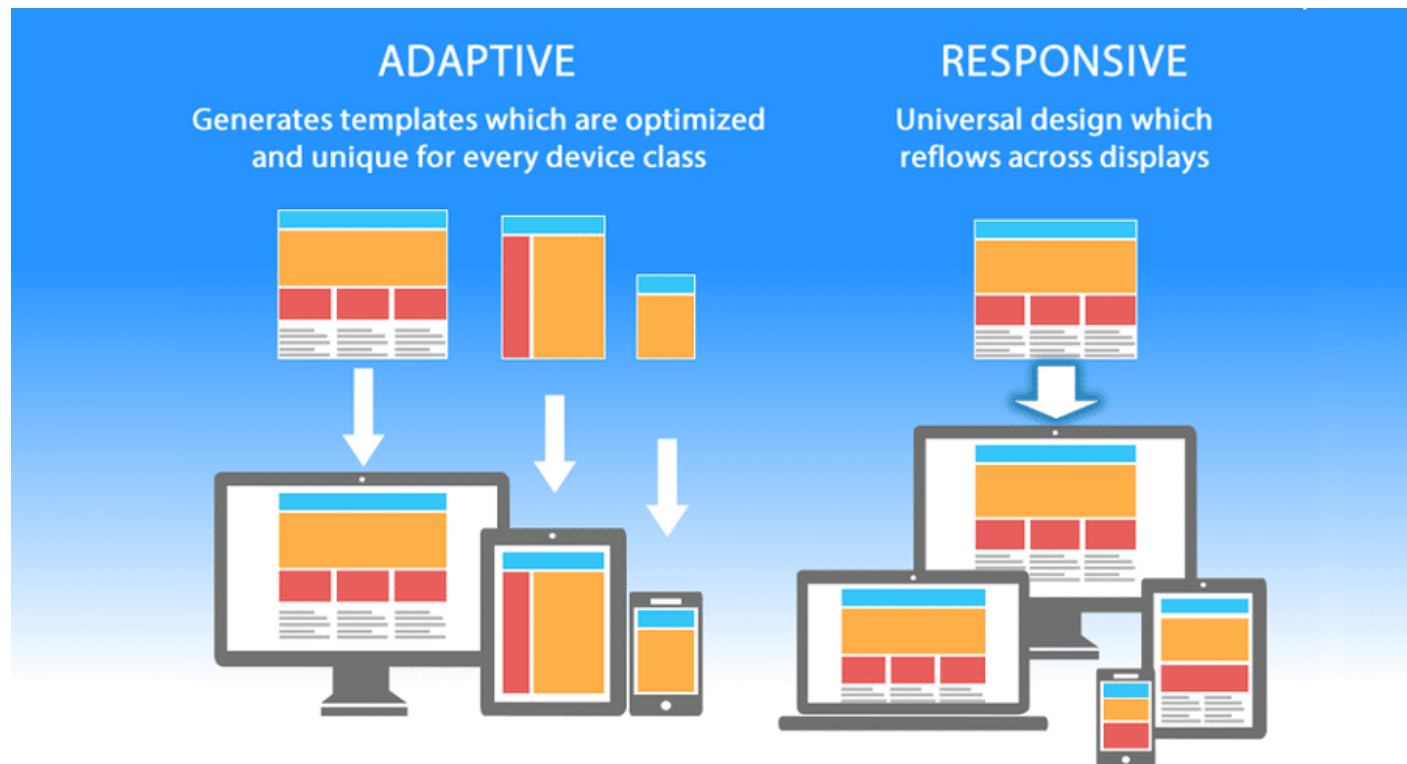
Това, което работи в тестовите среди, не винаги се задържа в реалния свят. Провеждането на тестове за употребимост на вашия продукт с реални потребители на реални устройства ще ви позволи да откриете проблеми с потребителския интерфейс, които трябва да бъдат решени преди пускането в експлоатация.

При проектирането за множество екрани и устройства най-добрата стратегия е да имате предвид потребителското изживяване. Като дизайнер на потребителски изживявания, трябва да оцените кога, къде и как продуктът ще бъде използван, за да оцените оптималното изживяване за потребителя. Независимо от размера на екрана на вашия контент, потребителите очакват гладко изживяване на всички устройства.

II. Responsive design

A. Какво представлява Responsive Design?

Респонсивният дизайн е подход за уеб дизайн и разработка, който цели да осигури оптимално изживяване за потребителите на уебсайтове независимо от устройството, което използват за достъп - например компютър, таблет или мобилен телефон. Той използва гъвкави мрежи, адаптиращи се изображения и CSS медиязапитвания, за да реагира и да се адаптира към различните размери на екраните и устройствата.



Със създаването на респонсивен дизайн, уебсайтът се променя динамично, така че да изглежда и функционира оптимално както на големи екрани, така и на по-малки устройства, като се запазва добро разбиране и лесно навигиране за потребителите. Това може да включва пренареждане на съдържанието, скриване на ненужни елементи, увеличаване на текста и други техники, които да подобрят изживяването на потребителя.

Резултатът е, че респонсивният дизайн осигурява постоянно и приятно изживяване за потребителите, като се гарантира, че уебсайтът се приспособява към различните устройства и размери на екраните, които потребителите могат да използват.

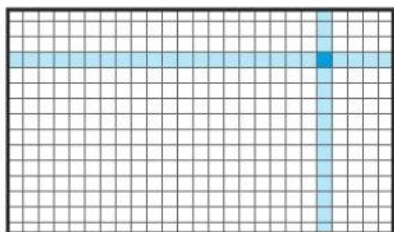
B. Основни принципи

Три основни принципа задвижват респонсивния дизайн. За някои дизайни може да влязат в игра и други принципи, но тези три свързват всички респонсивни сайтове:

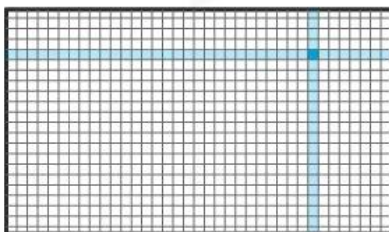
1) Fluid Grid System

Гъвкавата решетъчна система е един от основните принципи на респонсивния уеб дизайн. Тя представлява метод за организиране на съдържанието на уебсайта чрез използване на относителни, вместо абсолютни, размери за разпределение на елементите на страницата.

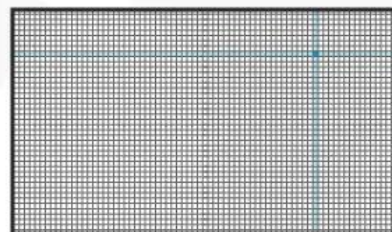
720p (HD)
1280 x 720 pixels



1080p (Full HD)
1920 x 1080 pixels



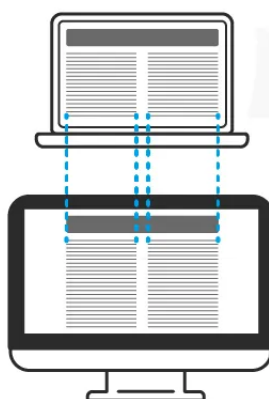
4K (Ultra HD)
3840 x 2160 pixels



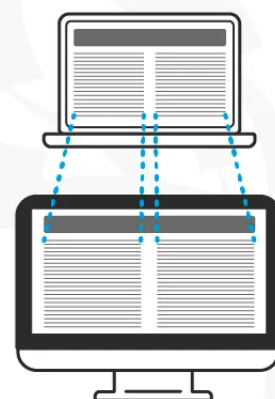
Вместо да определят фиксирани размери в пиксели за различните елементи на уебсайта, като например широчина на колони и отстъпи, гъвкавата решетъчна система използва проценти или други относителни единици за измерване. Това позволява на елементите на уебсайта да се разпределят динамично и да се приспособяват към различните размери на екраните на устройствата, като се поддържа хармонично и балансирано изживяване за потребителите.

Гъвкавата решетъчна система в респонсивния дизайн позволява на уебсайтовете да се скалират гладко и еластично, без да загубват функционалност или визуална привлекателност. Това включва автоматично пренареждане на колоните, отстъпите и другите елементи на страницата, така че да се запази доброто усещане за потребителите независимо от това дали те достъпват уебсайта от компютър, таблет или мобилен телефон.

Fixed Version



Fluid Version



Създаването на оформлениа и елементи, които се приспособяват към различни размери на екрана и устройства, е от съществено значение за постигането на респонсивен уеб дизайн. Респонсивните единици са измервания, използвани в уеб дизайна, които са от съществено значение за постигането на респонсивен уеб дизайн, като гарантират, че уебсайтовете изглеждат добре и функционират добре на

различни устройства, като настолните компютри, лаптопи, планшети и мобилни телефони. Ето някои от често използваните респонсивни единици:

- **Проценти (%)** - Използването на проценти позволява елементите да бъдат мащабирани спрямо техния родителски контейнер. Например, задаването на ширина от 50% за div контейнер означава, че той ще заеме половината ширина на своя родителски контейнер.
- **Viewport Width (vw) и Viewport Height (vh)** - Тези единици представляват процент от ширината или височината на прегледното пространство. Например, задаването на ширина на елемент от 50vw означава, че той ще заеме половината ширина на прегледното пространство. Подобно на това, използването на vh ще бъде процент от височината на прегледното пространство.
- **Минимална и максимална ширина и височина (min-width, max-width, min-height, max-height)** - Тези единици позволяват задаването на минимални и максимални размери за елементите. Например, задаването на max-width: 100% гарантира, че изображението няма да надвиши ширината на своя родителски контейнер.
- **ЕМ (em) и REM (rem)** - ЕМ и REM са относителни единици, базирани на размера на шрифта на родителския елемент (ЕМ) или корена (REM). ЕМ е относително спрямо размера на шрифта на родителския елемент, докато REM е относително спрямо размера на шрифта на кореновия HTML елемент. Използването на тези единици за типография и отстъпи/падинги позволява скалируемост към различни размери на екрана.
- **Viewport-Relative Lengths (vmin, vmax)** - Тези единици са свързани с по-малкото (vmin) или по-голямото (vmax) измерение на прегледното пространство, което помага за създаването на по-адаптивни дизайни.

Като използвате тези респонсивни единици в комбинация с медийни заявки и други техники за респонсивен дизайн, уеб разработчиците могат да създадат гъвкави и приспособими оформления и елементи, които реагират на устройството на потребителя, размера на екрана и ориентацията му. Това помага за създаването на по-потребителски приятно изживяване, като гарантира, че съдържанието е четимо, достъпно и визуално привлекателно на различни устройства.

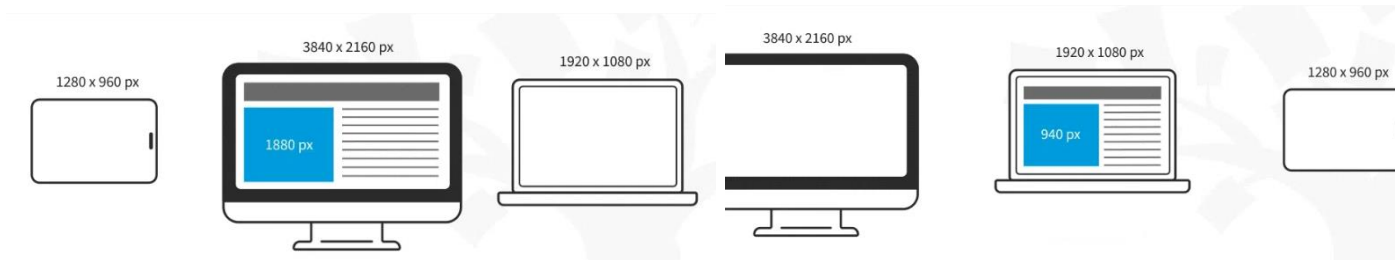
2) Fluid Image Use

Гъвкавото използване на изображения е важен аспект от респонсивния уеб дизайн. Когато използвате гъвкави решетъчни системи за дефиниране на макета си с относителни стойности, като проценти, размерът на всички елементи в макета, включително и изображенията, се променя спрямо размера на екрана на устройството.

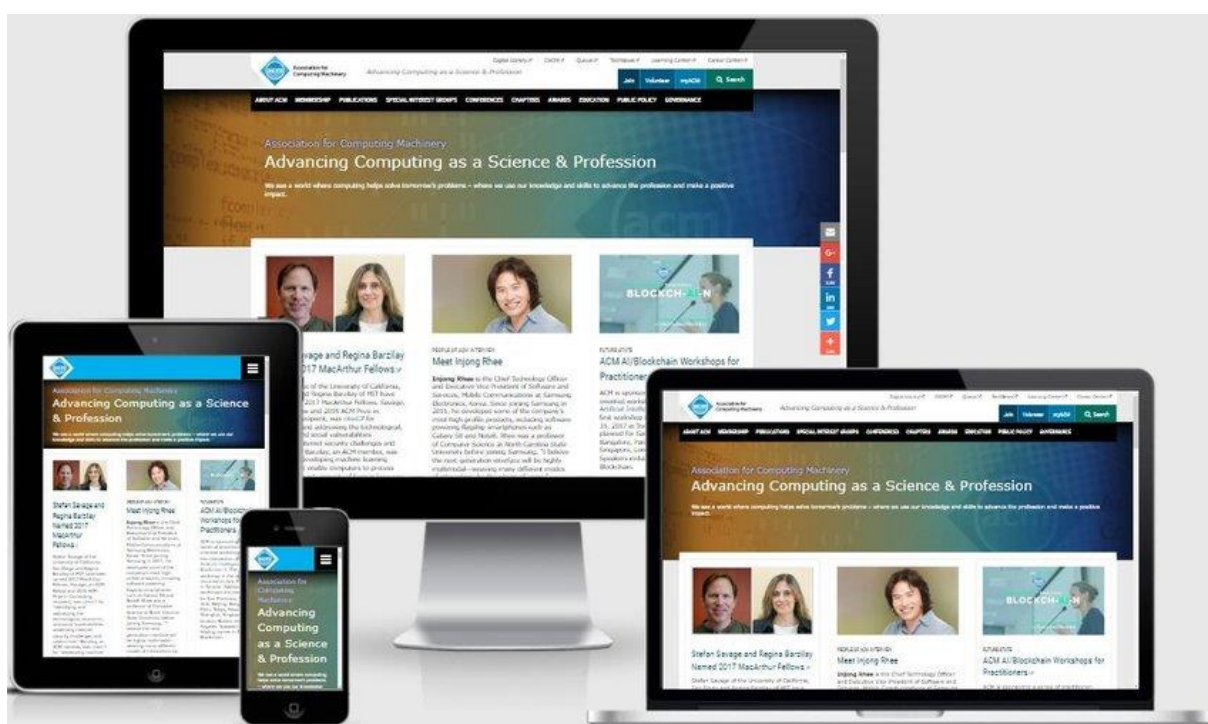
Това означава, че вместо да задавате фиксирани размери за изображенията, можете да ги настроите да се скалират автоматично спрямо размера на техния контейнер. Така, когато уебсайтът се зареди на устройство с по-голям екран, изображенията



ще бъдат показани с по-голям размер, а при по-малки екрани ще бъдат скалирани надолу, запазвайки съотношението и качеството си.



Гъвкавото използване на изображения не само осигурява оптимално изживяване за потребителите на различни устройства, но и помага за ускоряване на зареждането на уебсайта, като се намалява размерът на изображенията спрямо конкретните нужди на устройството. За да постигнете това, може да използвате различни техники, като например оптимизация на изображенията за уеб, използване на формати с по-добра компресия и други.



3) Media Queries

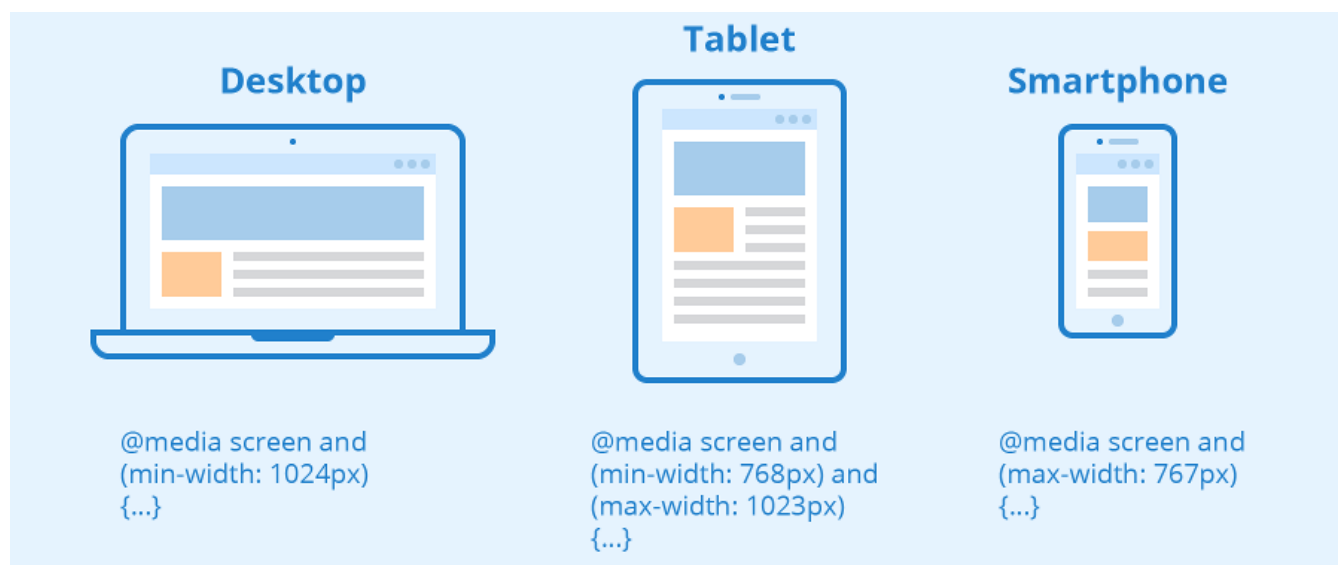
Медийните заявки са ключов инструмент в респонсивния уеб дизайн. Те позволяват на уебсайтовете да реагират динамично на различните размери на екраните на устройствата, като променят стиловете и подреждането на съдържанието в зависимост от определени условия.

Основната цел на медийните заявки е да предоставят инструкции на браузъра за това как да реагира на различни контексти на устройството, като например размерът на екрана, разделителната способност или ориентацията на устройството. Например, може да се поиска определен стил да се приложи, ако екранът е по-широк от определена ширина или ако устройството е в режим на пейзаж.

Един от най-честите начини за използване на медийните заявки е чрез определяне на "преградни точки" или "breakpoints". Това са конкретни ширини на екрана, при които макетът на уебсайта се

Проектиране на потребителски интерфейси

променя, за да се адаптира към различните размери на устройствата. Например, можете да дефинирате медийна заявка, която променя стиловете на уебсайта, когато екранът е по-малък от 768 пиксела, за да се осигури по-добро изживяване за потребителите на мобилни устройства.



Медийните заявки са основен инструмент за създаване на адаптивни и респонсивни уебсайтове, като осигуряват консистентно и удобно изживяване за потребителите независимо от техните устройства. За по-добри резултати, често се използва комбинация от медийни заявки, гъвкави решетъчни системи и гъвкаво използване на изображения, за да се постигне идеалното подреждане и стил за всяко устройство.

С. Най-добри практики

- **Мобилен подход (Mobile-First Approach):** Започнете да проектирате за мобилни устройства и постепенно подобрявайте оформлението за по-големи екрани. Това осигурява гладко изживяване за всички потребители и намалява времето за зареждане за мобилните посетители.
- **Използвайте CSS Frameworks:** Възползвайте се от CSS фреймуърк като Bootstrap или Foundation, които предлагат респонсивни решетки и компоненти от вкъщи, спестявайки време за разработка и гарантирайки съвместимост.
- **Респонсивни единици:** Чрез използването на тези респонсивни единици в комбинация с медийни заявки и други техники за респонсивен дизайн, уеб разработчиците могат да създадат гъвкави и приспособими оформления и елементи, които реагират на устройството, размера на екрана и ориентацията на потребителя.
- **Мащабируема типография:** Използвайте мащабируеми размери на шрифтове, за да поддържате четливостта и разбираемостта на различните устройства и размери на екрана.
- **Медийни заявки:** Създайте медийни заявки, които насочват различни размери на екрана. Тези заявки ви позволяват да регулирате размерите на шрифтове, отстъпите и вътрешните отстъпи, за да гарантирате, че съдържанието ви остава четимо и визуално приятно.

- **Viewport Meta Tag:** Включете viewport meta тага във вашия HTML, за да инструктирате брауъра как да мащабира уеб страницата, за да се побере на екрана. Този таг е съществен за мобилна оптимизация.
- **Гъвкави изображения:** Задайте max-width: 100% на изображенията, за да предотвратите преливането им извън контейнерите и гарантирате, че те се мащабират пропорционално на размера на екрана.
- **Приоритизиране на съдържанието:** Обмислете кое съдържание е най-важно и гарантирайте, че то остава видимо на по-малки екрани. Може да е необходимо да приоритизирате съдържанието и да пренареждате елементите.
- **Тестване:** Редовно тествайте вашия уебсайт на различни устройства и брауъри, за да идентифицирате и отстраните респонсивни проблеми. Емулятори, симулатори и инструменти за разработчици на брауъри са полезни за тази цел.
- **Оптимизация на производителността:** Уверете се, че вашият респонсивен дизайн не компрометира времето за зареждане на страницата. Оптимизирайте изображенията, минимизирайте HTTP заявките и използвайте кеширане на брауъра.
- **Потребителско изживяване:** Не се фокусирайте само върху оформлението; обмисляйте цялостното потребителско изживяване. Докосваема навигация, четими шрифтове и логичен поток на съдържанието са съществени за мобилните потребители.

Респонсивният уеб дизайн е от съществено значение в съвременната цифрова среда, като обслужва разнообразната гама от устройства и потребителски предпочитания. Следването на тези принципи и най-добри практики може да помогне за създаването на уебсайтове, които предлагат последователно и потребителски удобно изживяване на различни платформи и размери на екрана.

В заключение, респонсивният уеб дизайн не е опция; той е основно изискване за уеб разработката в съвременната разнообразна цифрова среда. С приветстване на гъвкавите оформления, гъвкавите изображения и медийните заявки, и следване на най-добрите практики, можете да създадете уебсайтове, които предоставят безпроблемно изживяване на широк спектър от устройства. Бъдете информирани за последните тенденции и технологии в респонсивния дизайн, за да осигурите, че вашите уебсайтове остават приспособими и потребителски удобни в постоянно развиващия се цифров свят.

IV. Съвременни средства за реализация на потребителски интерфейси

A. React - общ преглед, основни принципи и примери за използване

1) Общ преглед

React (известен също като React.js или ReactJS) е JavaScript библиотека за изграждане на

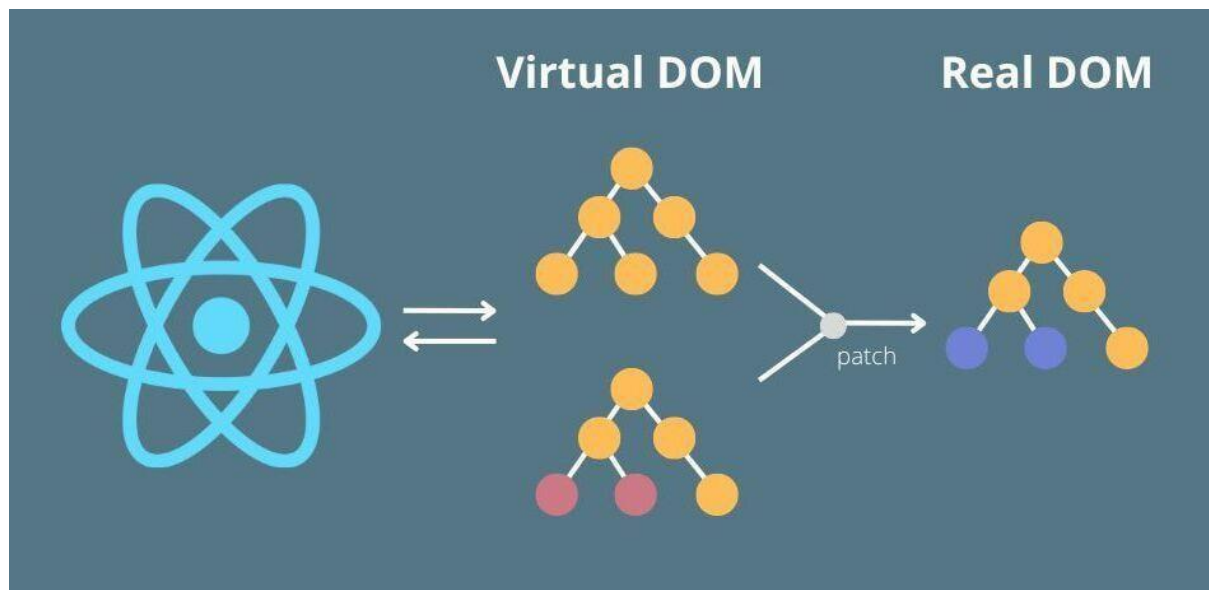


потребителски интерфейс, с която може да се програмират уебсайтове и мобилни софтуерни приложения, която е с отворен код. Поддържа се от Facebook и общност от разработчици и компании.

Един от основните принципи на React е компонентният подход, който позволява разделение на потребителския интерфейс на независими и преизползваеми компоненти. Това прави кода по-структуриран, лесен за поддръжка и разширение.

2) Основни принципи

- **Virtual DOM:** React използва виртуален DOM, който е абстракция над реалния DOM. Това позволява ефективно управление на промените в интерфейса и оптимизира производителността.

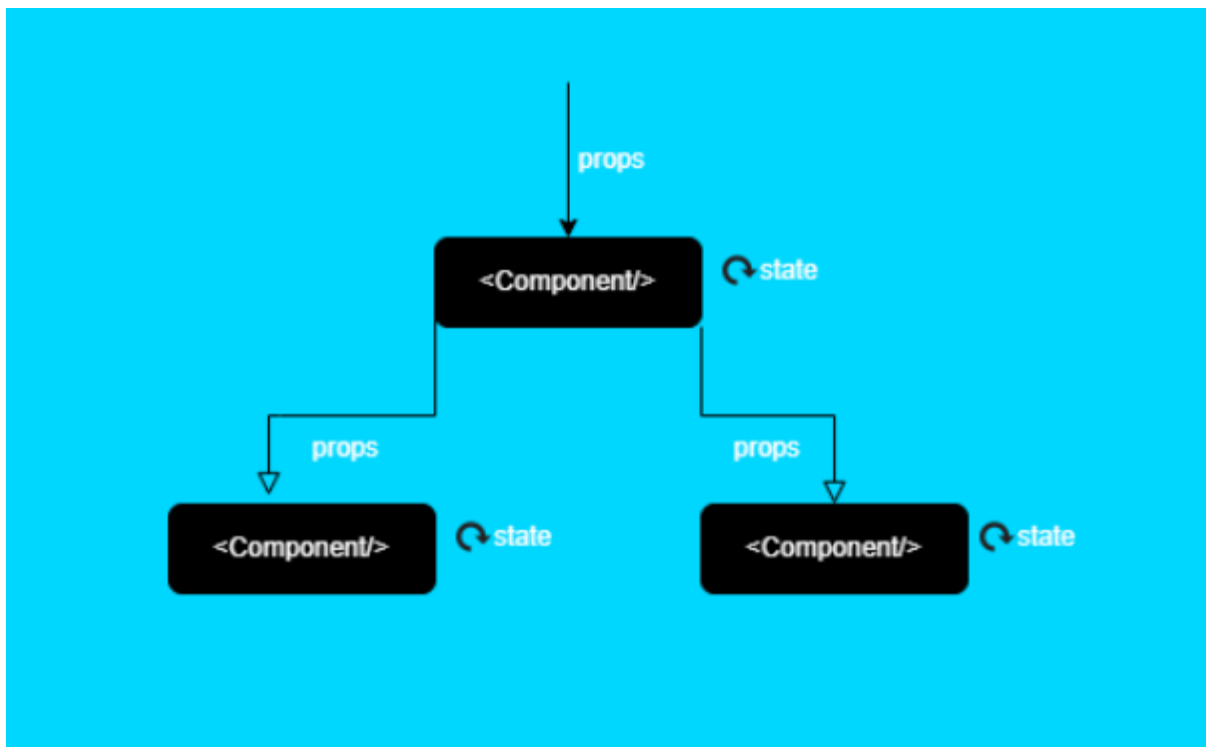


DOM (Document Object Model) е структура за представяне на HTML или XML документ в браузъра. Той се състои от различни елементи като тагове, атрибути и текстове, които формират йерархично дърво. Когато браузърът зарежда уеб страница, той създава DOM от HTML кода и го използва за показване на съдържанието на страницата на потребителя.

Virtual DOM (виртуален DOM) е абстракция над реалния DOM, която се използва от React. Той е вътрешна репрезентация на DOM структурата в паметта на браузъра. Когато се правят промени върху състоянието на приложението в React, тези промени се отразяват върху виртуалния DOM, а не директно в реалния DOM. След това React използва алгоритми за сравнение (diffing algorithms), за да сравни виртуалния DOM с реалния DOM и да открие разликите между тях.

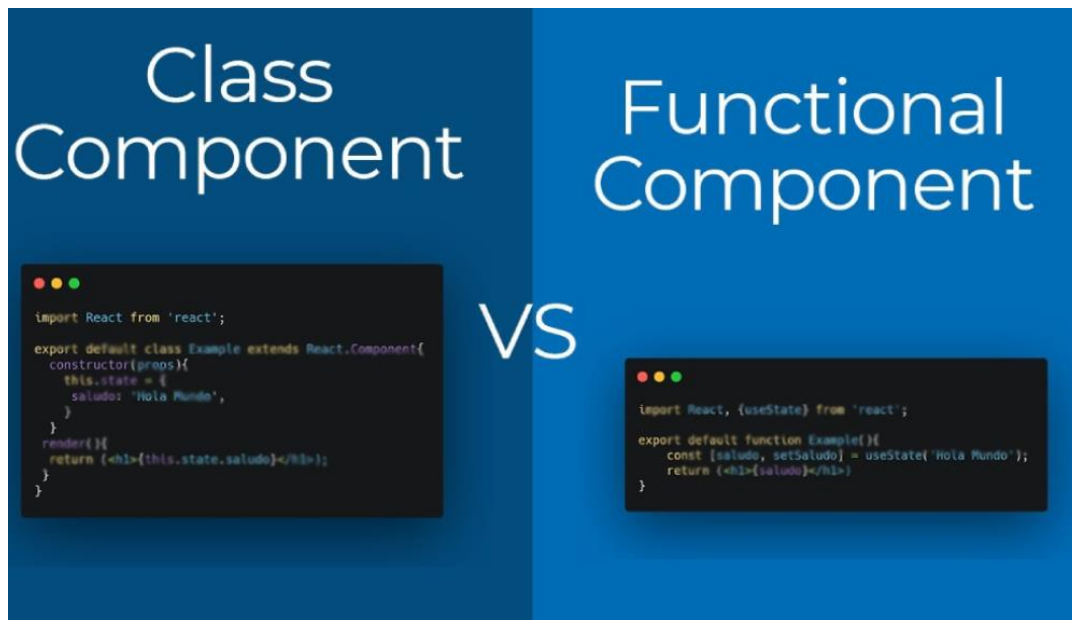
Разликата между виртуалния DOM и реалния DOM е, че виртуалният DOM се намира в паметта на браузъра и е много по-бърз за манипулиране, докато реалният DOM се намира в браузъра и манипулациите върху него са по-бавни. Като използва виртуалния DOM, React може да оптимизира процеса на прерисуване на интерфейса и да предостави по-голяма производителност, като само обновява частите на интерфейса, които са променени, вместо да прерисува целия интерфейс. Това води до по-бързо и по-ефективно управление на промените в интерфейса и по-добра производителност на уеб приложението.

- **Компоненти:** Всеки интерфейсен елемент в React се представя като компонент. Компонентите могат да бъдат класови или функционални.



Компонентите в React са основни строителни блокове на уеб приложенията. Те представляват изолирани, многократно използвани елементи на потребителския интерфейс, които се използват за изграждането на цялата визуална част на уеб приложението. В React, всеки интерфейсен елемент, като заглавие, бутон или форма, се представя като компонент.

Има два основни типа на компоненти в React:



Функционални компоненти

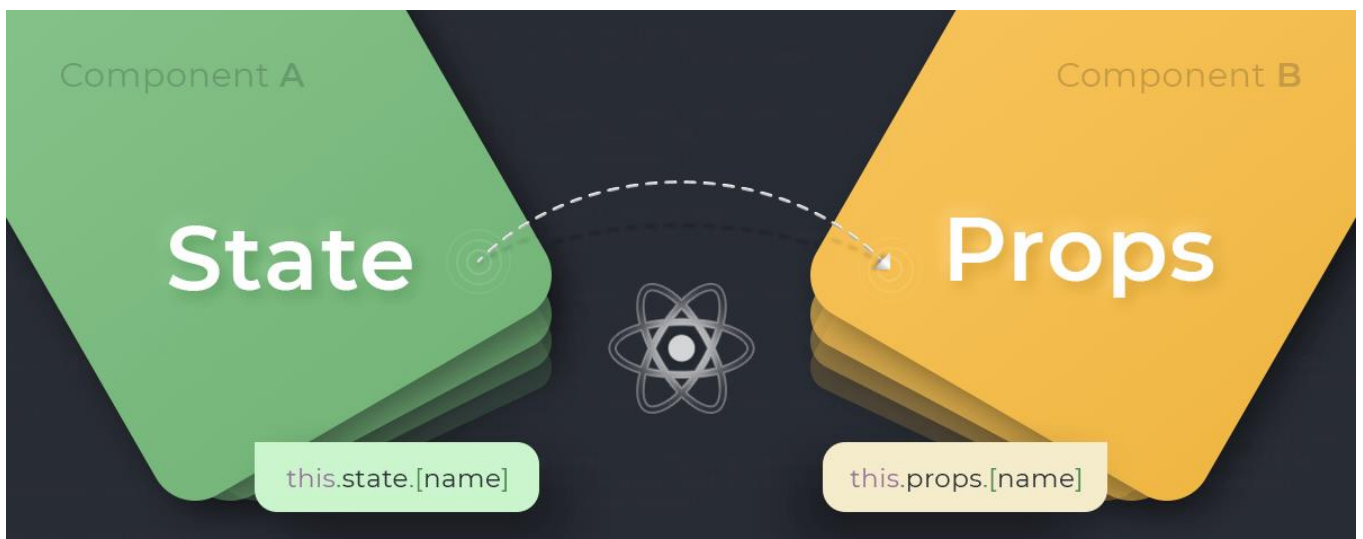
- Те са прости функции в JavaScript, които приемат пропс като параметър и връщат React елемент (често чрез JSX).
- Използват се за прости компоненти без вътрешно състояние или методи на жизнен цикъл.

Класови компоненти

- Те са ES6 класове, които наследяват от `React.Component`.
- Позволяват съхранение на вътрешно състояние и имат методи на жизнен цикъл, които се изпълняват в различни точки на живота на компонента.

Компонентите позволяват разделянето на интерфейса на по-малки, независими части, които могат да се преизползват и да се поддържат по-лесно. Те образуват дървовидна структура на компоненти, като всяка част от дървото представлява различна част от потребителския интерфейс. Това прави управлението на сложни приложения много по-лесно и организирано. Компонентите в React са една от ключовите абстракции, които правят тази библиотека толкова мощна и популярна за създаване на уеб приложения.

- **Състояние (State):** React позволява деклариране и управление на състоянието на компонентите. Състоянието определя какво трябва да се показва в интерфейса и какво трябва да се случи при промяна.
- **Пропс (Props):** Пропс са параметри, които се предават на компоненти и могат да бъдат използвани за настройка и пренос на данни между компоненти.

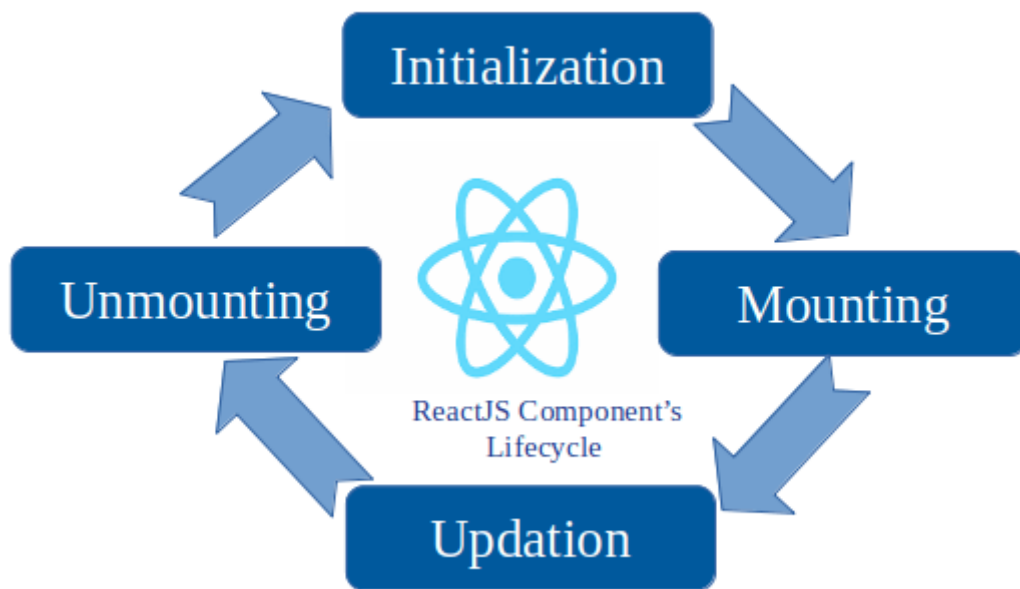


Състоянието (State) в React е механизъм, който позволява на компонентите да съхраняват и управляват данни, които могат да се променят с времето. Това състояние определя какво трябва да се показва в интерфейса на компонента и какви действия или промени трябва да се случат при взаимодействие с потребителя или други части на приложението. Например, в компонент за брояч, състоянието би било текущата стойност на брояча, а при кликане на бутон за увеличаване или намаляване на брояча, състоянието се променя, което води до актуализация на интерфейса, за да отрази новата стойност на брояча.

Пропс (Props) са параметри, които се предават на компонентите и които могат да бъдат използвани за настройка и пренос на данни между компоненти. Тези данни се предават от родителския компонент на децата му и могат да бъдат използвани в децата по различни начини, например за показване на информация или за задаване на определени настройки. Props са без изменяеми (immutable), което означава, че компонентите не трябва да променят техните props директно; вместо това, те трябва да се считат за read-only и да бъдат използвани само за показване на информация или за задаване на конфигурация на компонентите. Например, можем да предадем

на компонент за заглавие пропс със стойност "Добре дошли" и този компонент ще го покаже в своя интерфейс. Props позволяват на компонентите да бъдат гъвкави и преносими, като по този начин ни позволяват да създадем мощни и многократно използвани интерфейсни елементи в нашите приложения.

- **Жизнен цикъл (Lifecycle):** React предоставя методи за управление на жизнения цикъл на компонентите, като `componentDidMount`, `componentDidUpdate` и `componentWillUnmount`.



Жизненият цикъл на компонентите в React представлява поредица от етапи, през които преминават компонентите от създаването им до тяхната демонтаж. React предоставя различни методи на жизнения цикъл, които позволяват на разработчиците да изпълняват код в определени точки от живота на компонента.

Ето някои от най-важните методи на жизнения цикъл в React:

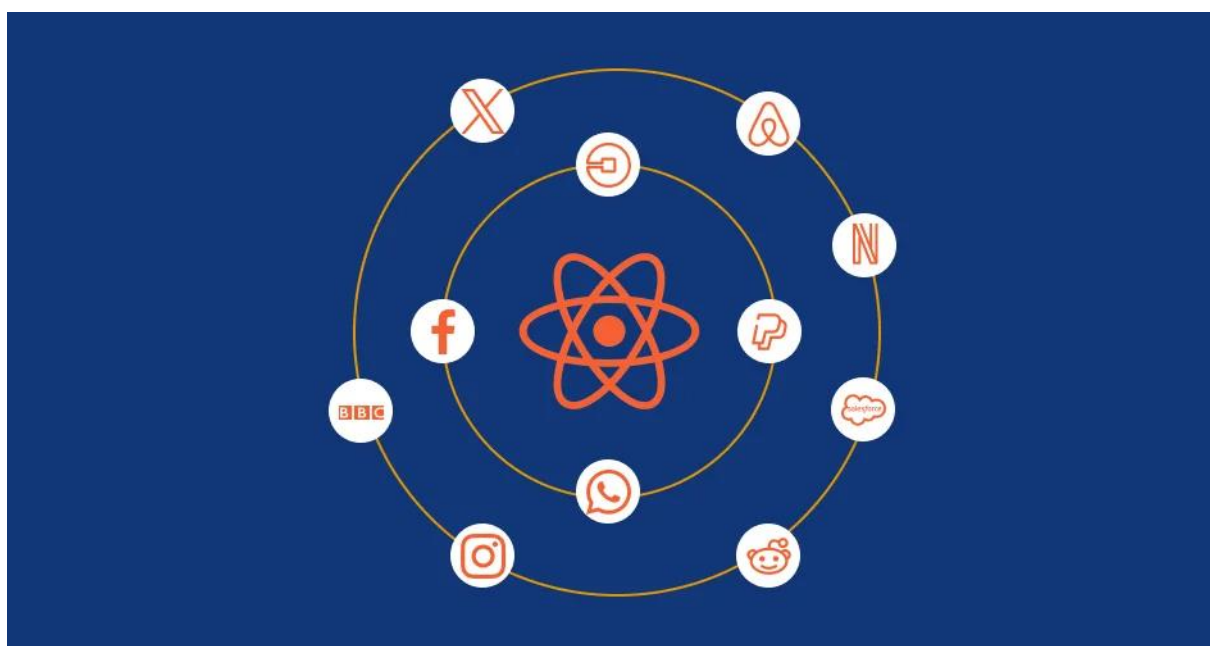
- **`componentDidMount()`** - Този метод се извиква, когато компонентът е първоначално монтиран в DOM-а. Той е подходящ за извличане на данни от външни източници, като например заявки към сървъри, инициализация на таймери или абонаменти към събития.
- **`componentDidUpdate(prevProps, prevState)`** - Този метод се извиква, след като компонентът е обновен и промените са били рефлексирани в DOM-а. Той позволява изпълнението на код, който зависи от предишното състояние или пропси на компонента.
- **`componentWillUnmount()`** - Този метод се извиква, преди компонентът да бъде демонтиран и премахнат от DOM-а. Той е подходящ за освобождаване на ресурси, като например прекъсване на таймери или анулиране на абонаменти към събития.

Тези методи на жизнения цикъл позволяват на разработчиците да извършват дейности, свързани с живота на компонентите, като се управлява тяхната инициализация, обновяване и демонтаж. Те са полезни за изпълнение на различни действия в зависимост от текущото състояние на компонентите и помагат за оптимизиране на управлението на паметта и ресурсите във вашите приложения.

3) Примери

Ето няколко примера за уеб приложения и уеб страници, които използват React:

- Facebook - Цялата уеб платформа на Facebook използва React. Това включва началната страница, профилите на потребителите, новините, съобщенията и други функционалности.
- Instagram - Instagram използва React за построяване на своя уеб интерфейс. Това включва профилната страница, потока със снимки, търсенето и други части на уебсайта.
- Netflix - Netflix използва React за своя уеб интерфейс, като включва началната страница, категориите с филми и сериали, търсенето и др.
- WhatsApp Web - Уеб версията на WhatsApp използва React за изграждане на своя интерфейс, включително списъка с чатове, съобщенията, настройките и др.
- GitHub - GitHub използва React за своя уеб интерфейс, който включва репозиториите, прегледа на кода, проблемите, пул реквестите и други части на платформата.



Тези са само някои от множеството уеб приложения и страници, които използват React за създаване на потребителски интерфейси с висока производителност и лесна поддръжка.

В. Vue - общ преглед, основни принципи и примери за използване

1) Общ преглед

Vue.js е модерен JavaScript framework за създаване на потребителски интерфейси и един от най-популярните инструменти за уеб разработка в момента.

Vue.js е лек, гъвкав и лесен за научаване framework за уеб разработка.

Той се фокусира върху изграждането на потребителски интерфейси и е подходящ за различни видове приложения - от едностранни приложения (SPA) до по-традиционни мулти-странични приложения.

Vue има активна общност и екосистема от разширения, което прави уеб разработката с него още по-привлекателна.



2) Основни принципи

Learning SOLID principles for better Vue 3 coding.



S - Single Responsibility
O - Open and Close
L - Liskov Substitution
I - Interface segregation
D - Dependency inversion

Принцип на единичната отговорност - Single Responsibility Principle (SRP) :

Принципът на единичната отговорност ни насочва към това да разделим нашите компоненти така, че всеки от тях да има само една задача или отговорност. В Vue, това може да се постигне като разделим нашите компоненти на по-малки и по-специфични части, които се грижат за определени аспекти на приложението. Така създаваме компоненти като ProductList, ProductFilter и ProductDetail, които имат ясно дефинирани отговорности и са по-лесни за поддръжка и разширение.

Принцип на отворената/затворената функционалност - Open/Closed Principle (OCP):

Принципът на отворената/затворената функционалност ни насърчава да проектираме нашите компоненти така, че да са отворени за разширение, но затворени за промяна. Във Vue, това може да се постигне чрез използване на композиция и създаване на компоненти, които се комбинират за добавяне на нови функционалности, без да се налага промяна на съществуващия код.

Принцип на заместваемостта на Лисков - Liskov Substitution Principle (LSP):

Принципът на заместваемостта на Лисков насочва към това да проектираме нашите компоненти по такъв начин, че да могат да бъдат заместени с по-специфични или разширени версии, без да се нарушава функционалността на програмата. В Vue, това може да се постигне чрез използване на наследяване и полиморфизъм, където по-специфични компоненти могат да наследят и разширят функционалността на по-обща компоненти.

Принцип на разделянето на интерфейса - Interface Segregation Principle (ISP)

Принципът на разделянето на интерфейса насочва към това да разделим големите и сложни интерфейси на по-малки и по-специфични интерфейси, които обслужват конкретни нужди на клиентите. В Vue, това може да се постигне чрез разделяне на големите компоненти на по-малки и по-специфични компоненти, които обслужват конкретни части от потребителския интерфейс.

Принцип на инверсията на зависимостта - Dependency Inversion Principle (DIP):

Принципът на инверсията на зависимостта насочва към това да проектираме нашите компоненти така, че високо ниво модули да не зависят директно от ниско ниво модули, а отношенията да се базират на абстракции. В Vue, това може да се постигне чрез използване на инжектиране на зависимости и създаване на слаби връзки между различните части на приложението.

3) Примери

Ето някои известни уеб страници, които използват Vue.js и предоставят примери за неговото използване:

- GitHub - GitHub използва Vue.js за множество от своите страници и функционалности, като например GitHub Issues и GitHub Marketplace. GitHub е добър пример за използване на Vue.js за създаване на сложни уеб приложения.
- Facebook - Facebook използва Vue.js в различни части от своята уеб платформа. Например, Facebook Messenger е изграден с Vue.js. Той е добър пример за използване на Vue.js за създаване на бързи и отзивчиви приложения за чат.
- Netflix - Някои части от уеб страницата на Netflix са създадени с Vue.js. Въпреки че Netflix не е напълно базиран на Vue.js, те използват Vue.js за рендиране на определени компоненти и функционалности.
- Adobe - Adobe използва Vue.js в различни части от своите уеб приложения, включително Adobe Portfolio и Adobe Spark. Това са примери за използване на Vue.js за създаване на креативни и интерактивни приложения.

Тези са само някои от примерите за използване на Vue.js в известни уеб страници. Vue.js е популярен избор за създаване на съвременни и отзивчиви уеб приложения, поради своята лекота за усвояване и гъвкавост.



C. Angular - общ преглед, основни принципи и примери за използване

1) Общ преглед

Angular е силно разпространен уеб фреймуърк, разработен и поддържан от екипа на Google.

Angular е пълноценен уеб фреймуърк за разработване на съвременни уеб приложения и е известен със своите мощни възможности и широка общност от разработчици.

Той предлага комплексен набор от функции, включително директиви, компоненти, услуги, рутиране, форми и много други.

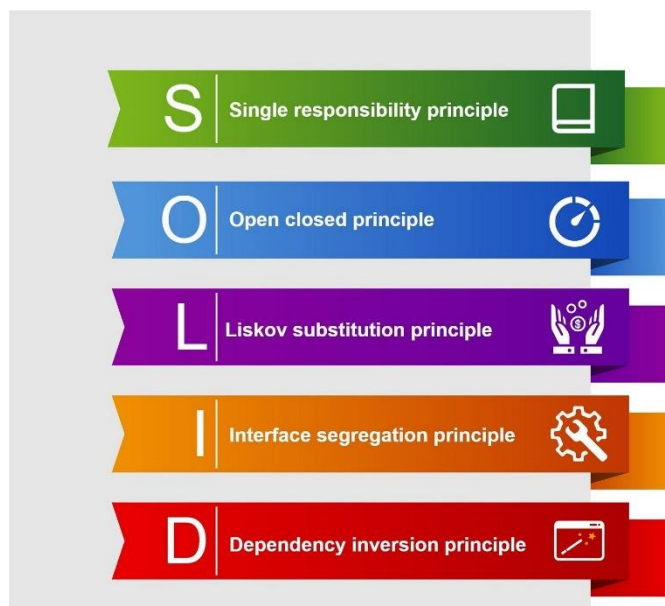
Angular следва принципа на компонентна архитектура, където приложението е разделено на множество компоненти, които се състоят от HTML шаблони, TypeScript код и стилове CSS.



2) Основни принципи

Принцип на единичната отговорност (Single Responsibility Principle - SRP)

В Angular компонентите трябва да имат само една причина за промяна. Те трябва да се фокусират върху конкретна функционалност и да не се опитват да изпълняват твърде много задачи. Например, компонент, отговарящ за показване на данни, не трябва да се смесва с логиката за тяхното променяне.



Принцип на отвореност/затвореност (Open/Closed Principle - OCP)

В Angular следва да създаваме модули, които са отворени за разширение, но затворени за модификация. Използването на модулна архитектура като модулите, услугите и компонентите позволява на приложението да бъде разширявано чрез добавяне на нови функции, без да се налага да се променя съществуващият код.

Принцип на подстановяването на Лусков (Liskov Substitution Principle - LSP)

В Angular класовете и интерфейсите следва да бъдат проектирани така, че да може да се извършва подмяна на екземпляр на даден клас с екземпляр на негов подклас, без да се променя очакваното поведение. Например, ако имаме базов компонент, очакваме всички негови наследници да могат да се използват безпроблемно във всички места, където се използва базовият компонент.

Принцип на разделянето на интерфейса (Interface Segregation Principle - ISP)

В Angular интерфейсите следва да бъдат разделени на по-малки и специфични интерфейси, които са фокусирани само върху конкретните нужди на тези, които ги използват. Това помага да се избегне зависимост на компоненти или услуги от методи, които те не използват.

Принцип на инверсията на зависимостта (Dependency Inversion Principle - DIP)

В Angular следва да създаваме код, който зависи от абстракции, а не от конкретни имплементации. Това означава, че компонентите и услугите следва да работят с интерфейси и абстракции, вместо да са напълно зависими от конкретни класове. Това гарантира по-голяма гъвкавост и лесно подменяне на части на приложението без промяна на целия код.

Като се прилагат тези принципи в Angular приложенията, можем да създадем по-модулни, поддържаеми и разширяеми системи, които са по-устойчиви към промени и по-лесни за разбиране и разработване.

3) Примери

Ето няколко примера за използване на Angular в известни платформи:

- **Google** - Angular е разработен от екипа на Google и се използва широко вътрешно за различни продукти и услуги на компанията. Някои от най-известните приложения, изградени с Angular от Google, включват Google Analytics и Google Cloud Platform Console.
- **YouTube TV** - YouTube TV, което е телевизионна услуга за стрийминг на живо, е изградено с Angular. Платформата осигурява достъп до телевизионни канали през интернет, използвайки Angular за своя фронтенд.
- **Forbes** - Един от най-големите медийни портали, Forbes, използва Angular за създаването на своя уебсайт. Angular помага за създаването на динамични и лесни за управление потребителски интерфейси, които са от съществено значение за медийния бранш.
- **Upwork** - Платформата за свободни проекти и наемане на фрилансъри Upwork използва Angular за създаването на своето потребителско преживяване. Angular помага за създаването на

интерактивни и атрактивни потребителски интерфейси, които улесняват процеса на търсене и наемане на фрилансъри.

- **PayPal** - Платформата за онлайн плащания PayPal използва Angular за създаването на потребителския си интерфейс както на уебсайта си, така и в мобилните си приложения. Angular предоставя инструменти за създаване на бързи и ефективни интерфейси за плащания, които са от съществено значение за успешното функциониране на платформата.



Тези са само няколко от множеството приложения и платформи, които използват Angular за създаване на своите потребителски интерфейси и приложения. Angular предоставя мощни инструменти за разработка, които са подходящи за различни видове проекти, от големи корпоративни приложения до малки уебсайтове и мобилни приложения.

Допълнителна информация за :

React -> <https://react.dev/>

Vue -> <https://vuejs.org/>

Angular -> <https://angular.io/>